

TurboGator

Adam Suma

Frederico Aberle

Jannis Alsbach

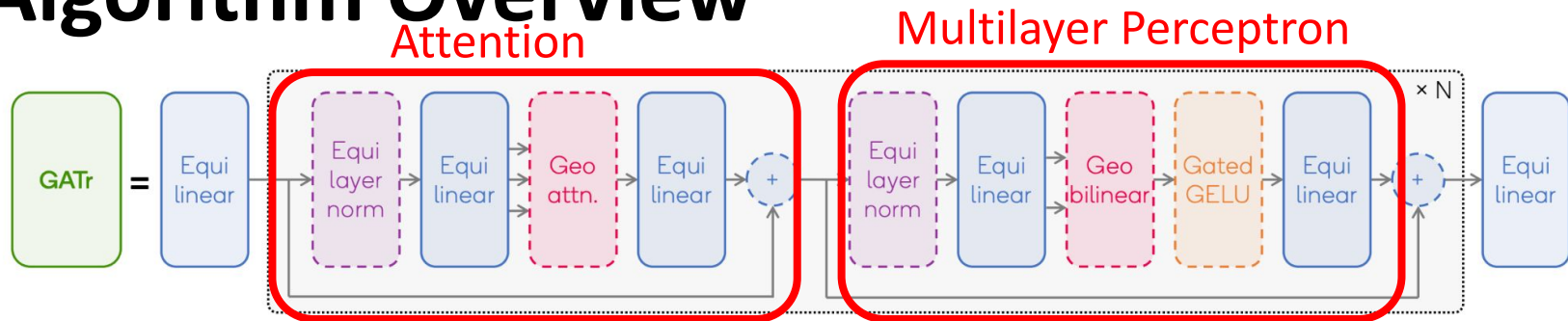
Mihai-George Licu



ETH

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Algorithm Overview



■ Initial embedding projection

■ Transformer Blocks

- MVOnlyGATrAttention
- MVOnlyGATrMLP

■ Out Projection

MVOnlyGATrModel(8, N, 2, 16)

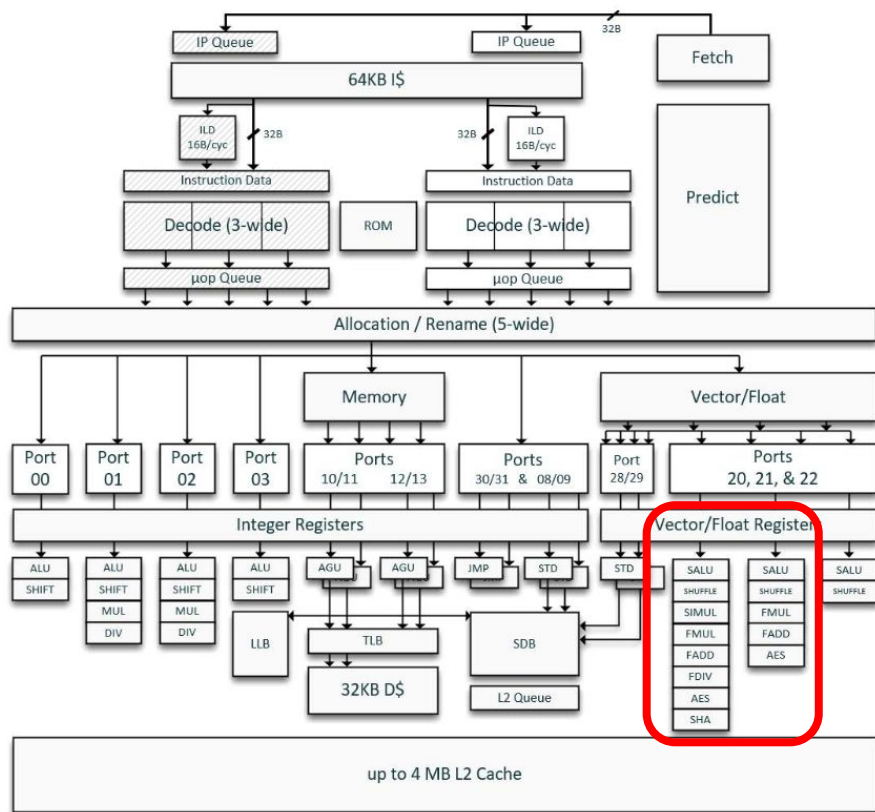
8 -> batch size

2 -> channels

16 ->



Server setup



- Intel N100 Alder Lake E core (Gracemont)
- Intel SSE4.1, Intel SSE4.2, Intel AVX2, FMA3
- Turboboost off 800MHz, 12 GB RAM
- 256b instructions get "cracked" in two 128b instruction

$\pi_s = 3.33$ flops/cycle (VFMADD132SD TP 1.66)

1.66 instr/cycle * 2 FLOPs/instr

$\pi_v = 16$ flops/cycle (VFMADD132PS YMM TP 1)

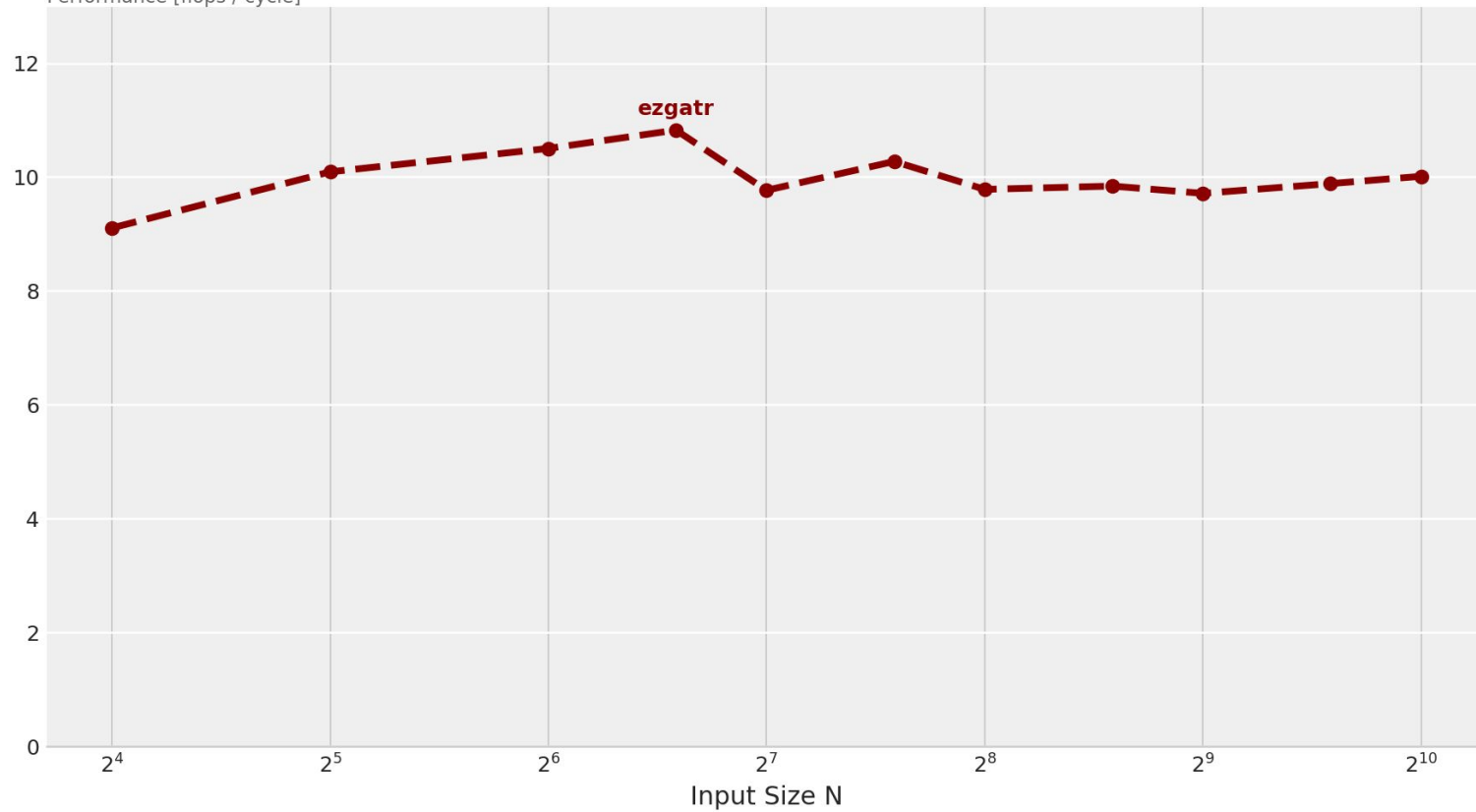
1 instr/cycle * 2 FLOPs/instr * 8 elements/vector

(XMM is 1.66?)

$\beta = 32$ bytes/cycle

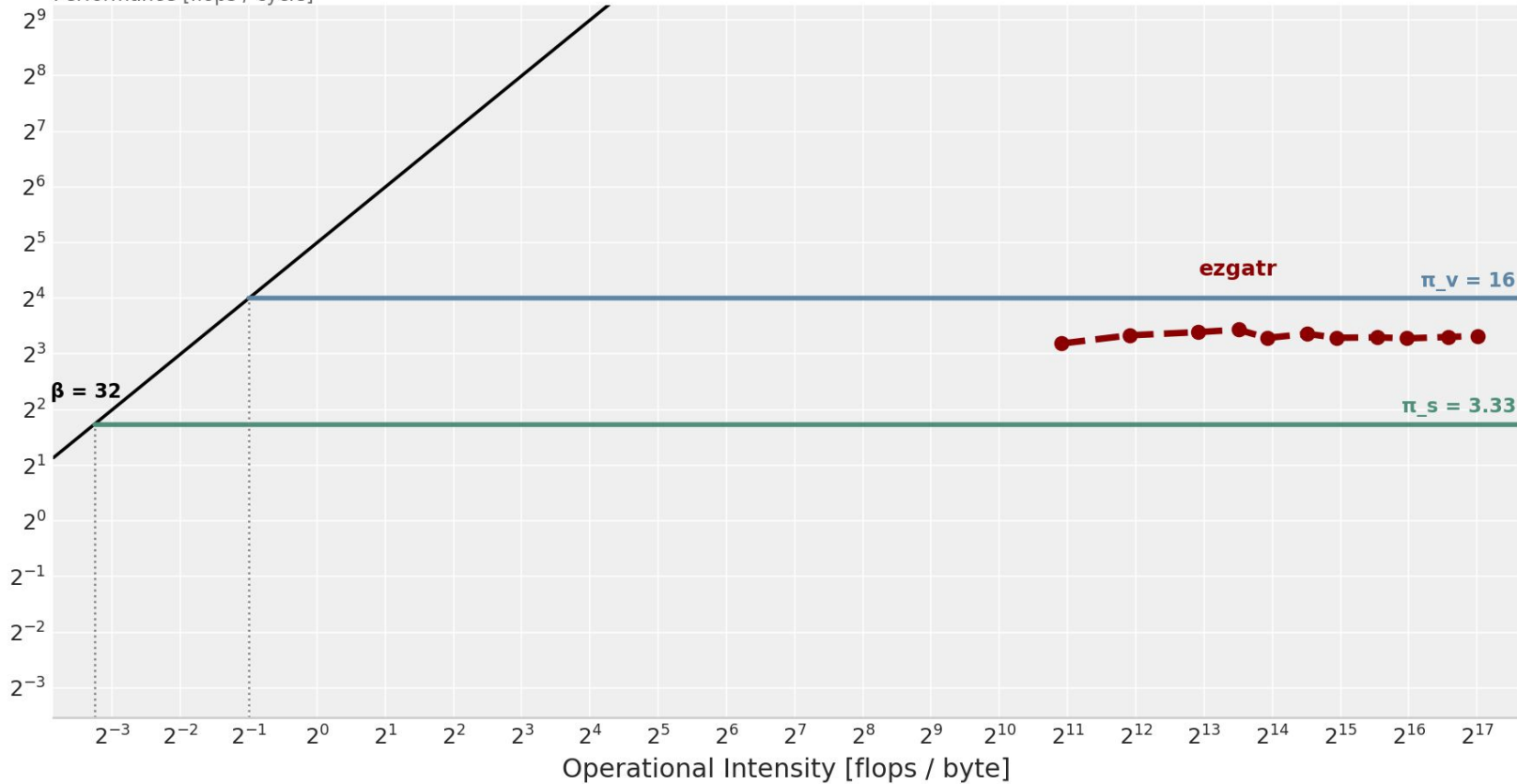
Performance: Intel N100 (1 Core, 800Mhz)

Performance [flops / cycle]



Roofline Model: Intel N100 (1 Core, 800Mhz)

Performance [flops / cycle]



FLOPs and Data Movement

N	$W(N)$	$Q(N)$
16	6,677,668,864	3,477,124
32	13,473,039,360	3,501,700
64	27,416,888,320	3,550,852
96	41,831,547,904	3,600,004
128	56,717,018,112	3,649,156
192	87,900,390,400	3,747,460
256	120,967,005,184	3,845,764
384	192,749,962,240	4,042,372
512	272,065,889,280	4,238,980
768	453,296,653,312	4,632,196
1024	664,659,297,280	5,025,412

Preliminary:

- $W(n) \rightarrow$ quadratic
- $Q(n) \rightarrow$ linear

Possible bottlenecks

Python:

- Dense matrix multiplication on sparse data
- General-purpose linear algebra libraries not optimized for geometric algebra

C++:

- Attention block bounded by memory-bandwidth
- Suboptimal vectorization by the compiler
- Bindings between Pytorch and C++ introduce unnecessary memory allocations, tensor copying, or data layout transformations

Ideas for optimisation

■ Sparsity

- bk, bq in `_compute_daa_qk_basis` mostly zeros.

■ Simplicity

- Similar to the previous point in the same basis

■ Vectorization

- SIMD instructions 8-way

How we plan on splitting up the work

- `EquiLinear` ← Jannis
- `EquiRMSNorm` ← to be determined based on effort
- `equi_geometric_attn()` ← Adam
- `scaler_gated_gelu()` ← Freddy
- `equi_join()` ← to be determined based on effort
- `geometric_product()` ← Mihai

Questions

- Do we run with python wrappers or do we run everything in C? How to profile python code?
- We assume $W(n)$ formula is static? How do we determine $Q(n)$ formula automatically?
- Doesn't python mess up with the caches?
- if $W(n)$ and $Q(n)$ stay static, and only thing that changes is cycles, what do we do?
- compare also against qualcomm implementation?
- How to calculate π_s and π_v , PS or PD? unequal FE and BE, cracks instr
- Beta L1, L2, L3, DRAM? where to even get all of these?
- Cold cache or warm cache? When? Why?
- should we count cached functions? @lru_cache
 - `@functools.lru_cache(maxsize=None, typed=True)`
 - `def compute_pin_equi_linear_basis (`